

Q1. Student Grade Book Application

CODE

```
students = {
    "Alice": [95, 92, 88, 91, 90],
    "Bob": [78, 82, 80, 76, 79],
    "Charlie": [65, 70, 68, 72, 69],
    "David": [55, 60, 58, 62, 57],
    "Eva": [85, 87, 90, 84, 88]
}

results = {
    name: {
        "marks": marks,
        "total": sum(marks),
        "average": sum(marks) / len(marks)
    }
    for name, marks in students.items()
}

def get_grade(avg):
    if avg >= 90:
        return "O"
    elif avg >= 80:
        return "A+"
    elif avg >= 70:
        return "A"
    elif avg >= 60:
        return "B+"
    elif avg >= 50:
        return "B"
    else:
        return "F"

for name in results:
    results[name]["grade"] = get_grade(results[name]["average"])

topper = max(results.items(), key=lambda x: x[1]["average"])

subject_means = []
for i in range(5):
    marks = [students[name][i] for name in students]
    subject_means.append(sum(marks) / len(marks))

highest_subject = subject_means.index(max(subject_means)) + 1

sorted_results = sorted(results.items(), key=lambda x: x[1]["average"], reverse=True)
```

Q1. Student Grade Book Application

CODE

```

students = {
    "Alice": [95, 92, 88, 91, 90],
    "Bob": [78, 82, 80, 76, 79],
    "Charlie": [65, 70, 68, 72, 69],
    "David": [55, 60, 58, 62, 57],
    "Eva": [85, 87, 90, 84, 88]
}

results = {
    name: {
        "marks": marks,
        "total": sum(marks),
        "average": sum(marks) / len(marks)
    }
    for name, marks in students.items()
}

def get_grade(avg):
    if avg >= 90:
        return "O"
    elif avg >= 80:
        return "A+"
    elif avg >= 70:
        return "A"
    elif avg >= 60:
        return "B+"
    elif avg >= 50:
        return "B"
    else:
        return "F"

for name in results:
    results[name]["grade"] = get_grade(results[name]["average"])

topper = max(results.items(), key=lambda x: x[1]["average"])

subject_means = []
for i in range(5):
    marks = [students[name][i] for name in students]
    subject_means.append(sum(marks) / len(marks))

highest_subject = subject_means.index(max(subject_means)) + 1

sorted_results = sorted(results.items(), key=lambda x: x[1]["average"], reverse=True)

print("=" * 70)
print("{<10} {<20} {<8} {<8} {<6}".format("Name", "Marks", "Total", "Average", "Grade"))
print("=" * 70)

for name, data in sorted_results:
    print("{<10} {<20} {<8} {<8.2f} {<6}".format(
        name,
        str(data["marks"]),
        data["total"],
        data["average"],
        data["grade"]
    ))

print("=" * 70)

print("\nClass Topper:")
print("Name :", topper[0])
print("Average :", round(topper[1]["average"], 2))
print("Grade :", topper[1]["grade"])

print("\nSubject Means:")
for i, mean in enumerate(subject_means, start=1):
    simatslab.saveetha.com/practice_app/classactivity/ajax/download_submissions.php

```

55/60

```
print("Subject", i, ":", round(mean, 2))
```

```
print("\nSubject with Highest Overall Mean : Subject", highest_subject)
```

OUTPUT

(no output)

Q2. Debugging and Rewriting

CODE

```

accounts = {}

def create_account(acc_no, name, balance):
    accounts[acc_no] = {'name': name, 'balance': balance, 'txns': []}

def deposit(acc_no, amount):
    if acc_no in accounts:
        accounts[acc_no]['balance'] += amount
        accounts[acc_no]['txns'].append(('Deposit', amount))

def withdraw(acc_no, amount):
    if accounts[acc_no]['balance'] >= amount:
        accounts[acc_no]['balance'] -= amount
        accounts[acc_no]['txns'].append(('Withdraw', amount))
    else:
        print("Insufficient balance")

def mini_statement(acc_no):
    for t in accounts[acc_no]['txns'][-5:]:
        print(t)

create_account(1001, "Ravi", 10000)
deposit(1001, 5000)
withdraw(1001, 20000)
withdraw(1001, 3000)
mini_statement(1001)
print("Balance:", accounts[1001]['balance'])

```

OUTPUT

(no output)